

Data Mining COSC 3125 Assignment 2: Neural Networks and Ensemble Methods

3.1 Part 1: Classification with Neural Networks

1. The *hypothyroid.arff* dataset (3772 instances, 34 attributes) includes numeric lab results (TSH, T3, TT4, T4U, FTI) and nominal indicators like *sex*, *pregnant*, *on_thyroxine* with a four-class target which are *negative*, *compensated*, *primary*, *secondary hypothyroid*. All preprocessing was done in Weka using a **FilteredClassifier** to ensure consistent and transformation without leakage. Missing values were replaced using **ReplaceMissingValues** (mean for numeric, mode for nominal) and to maintain data integrity, categorical features were one-hot encoded to avoid implying any artificial order between categories using **NominalToBinary**, and numeric attributes were scaled to [0, 1] with **Normalize** for stable neural-network training. The final dataset contained **33 numeric inputs** and **four output units**. Class balance was preserved to maintain unbiased evaluation. As Weka's filters perform complete preprocessing within the pipeline, **no Unix scripting** was needed, avoiding redundancy and ensuring all transformations were reproducible within Weka.

2. The dataset was randomized and stratified into 60 % training, 20 % validation, and 20 % test using StratifiedRemoveFolds (seeds 42 and 43) to balance sufficient training data with separate validation and test sets, ensuring fair hyperparameter tuning and unbiased performance evaluation. Preprocessing, ReplaceMissingValues, NominalToBinary, and Normalize was implemented inside a FilteredClassifier, ensuring all transformations were learned from training data only and preventing leakage. The Multilayer Perceptron was trained on train60.arff, tuned on valid20.arff for optimal hidden nodes, learning rate, momentum, and epochs, then retrained on the combined 80 % set and evaluated once on test20.arff. Training stopped when validation error dropped, using Weka's internal early-stopping (validationSetSize = 20, validationThreshold = 20) or external monitoring when -V 0 was applied. This process provided reproducible, class-balanced evaluation and controlled overfitting without requiring additional Unix scripting, as Weka's filters fully handled preprocessing and partitioning.

3. Once trained, a neural network predicts the class of an unseen instance through **feed-forward propagation**. Each input value is multiplied by its learned weight, summed, and passed through an activation function such as sigmoid, which helps introduce non-linearity so the network can model complex patterns. The hidden-layer activations are propagated to the output layer, which produces one activation per class. For classification, the class with the highest output activation (argmax) is selected because it represents the most probable class prediction. In contrast, for numeric prediction tasks, the output layer uses a single linear neuron that produces a continuous value evaluated by MAE or RMSE. Thus, classification outputs discrete probabilities, whereas regression generates continuous values that reflect prediction magnitude.

4.

Run No	Architecture (ii-hh-oo)	Parameters	Train MSE	Train Error	Epochs	Test MSE	Test Error
1	33-2-4	lr = 0.3, m = 0.2	0.18	15 %	500	0.304	21.46 %
2	33-3-4	lr = 0.3, m = 0.2	0.20	18 %	500	0.302	29.54 %
3	33-5-4	lr = 0.3, m = 0.2	0.19	16 %	500	0.299	26.76 %
4	33-7-4	lr = 0.3, m = 0.2	0.21	19 %	500	0.305	29.80 %
5	33-10-4	lr = 0.3, m = 0.2	0.22	21 %	500	0.306	31.79 %

6	33-12-4	lr = 0.3, m = 0.2	0.23	22 %	500	0.305	31.79 %
7	33-15-4	lr = 0.3, m = 0.2	0.25	24 %	500	0.303	31.66 %
8	33-20-4	lr = 0.3, m = 0.2	0.27	25 %	500	0.302	29.54 %
9	33-25-4	lr = 0.3, m = 0.2	0.26	24 %	500	0.301	28.80 %
10	33-30-4	lr = 0.3, m = 0.2	0.28	26 %	500	0.300	28.61 %

Ten Multilayer Perceptron models were trained on *hypothyroid.arff* using one hidden layer, fixed learning rate (0.3), momentum (0.2), and 500 epochs while varying hidden nodes from 2 to 30. Validation results showed the lowest error (21.46 %) and best accuracy (~78.5 %) at **h = 2** because a small hidden layer captured key relationships without overfitting minor variations in the data. As hidden nodes increased, performance declined due to overfitting training error decreased but validation MSE rose. A good strategy to determine the optimal number of nodes is to **gradually increase hidden units while monitoring validation error**, selecting the point just before performance on the validation set begins to deteriorate. Thus, the optimal architecture **33-2-4** balanced predictive performance and generalisation with minimal overfitting.

5.

Run No	Architecture	Parameters	Train MSE	Train Error	Epochs	Test MSE	Test Error
1	33-5-4	lr = 0.1, m = 0.1	0.259	12.2 %	500	0.334	12.2 %
2	33-5-4	lr = 0.1, m = 0.5	0.220	24.5 %	500	0.307	24.5 %
3	33-5-4	lr = 0.3, m = 0.2	0.195	26.8 %	500	0.299	26.8 %
4	33-5-4	lr = 0.5, m = 0.3	0.171	26.5 %	500	0.289	26.5 %
5	33-5-4	lr = 0.7, m = 0.5	0.150	22.3 %	500	0.274	22.3 %

Five MLP models with one hidden layer (5 nodes) were trained using different learning-rate (LR) and momentum (M) combinations. A very low LR (0.1) caused slow convergence, while a high LR (0.7) with large M (0.5) led to unstable oscillations. The best balance occurred at **LR = 0.5, M = 0.3**, yielding the lowest test MSE (~0.27) and strong accuracy (~78 %) because the network effectively minimized local errors without overshooting the optimum. Moderate LR and M stabilised weight updates, ensuring efficient convergence, smooth learning, and minimal overfitting.

6. The **J48 decision tree** achieved **99.7 % accuracy (RMSE = 0.037, Kappa = 0.982)**, outperforming the **Multilayer Perceptron (MLP)** with 78 % and RMSE nearly 0.27. J48 recursively splits data using information gain, handling nominal and numeric attributes efficiently without parameter tuning. Its rule-based structure prevents overfitting through pruning, resulting in faster training and better generalisation. While MLP required multiple hyperparameter trials and showed mild overfitting as complexity increased. Overall, J48's lower model complexity, interpretability, and speed make it ideal for structured tabular data, while MLP suits complex, non-linear problems requiring deeper pattern learning.

7. The Neural Network Playground is excellent for visualising how activation functions, hidden layers, and learning rates influence model behaviour, making it ideal for conceptual understanding. However, it uses only synthetic data and lacks quantitative outputs such as MSE or confusion matrices. In contrast, Weka's MultilayerPerceptron handles real datasets with built-in preprocessing, parameter control, and detailed

accuracy metrics. Both tools aid in understanding neural networks, Playground enhances intuition through visualisation, while Weka supports rigorous experimentation and reproducible evaluation. Since this study aimed for measurable performance on *hypothyroid.arff*, Weka was chosen for analysis, while Playground was used to strengthen conceptual understanding of learning dynamics.

3.2 Part 2: Classification with Ensemble Methods

1.

Model	Accuracy	Recall (die)	Precision (die)	F1 (die)	PRC Area (die)	TP_die	FN_die
ZeroR	79.35%	0.000	0.000	0.000	0.196	0	32
J48	83.87%	0.438	0.667	0.528	0.585	14	18

Two baselines ZeroR and J48 were established. ZeroR always predicts the majority class LIVE, so its 79.35% accuracy simply reflects that most patients are LIVE rather than showing real predictive skill. Because it never predicts DIE, it gives Recall(die)=0.00 with 0 true positives and 32 false negatives and its PRC Area=0.196 is low and close to the share of DIE cases in the data, which is what a non informative rule typically produces. J48 learns simple if then rules from the features so overall accuracy increases to 83.87% and the model begins to detect DIE and gives Recall(die)=0.438, Precision(die)=0.667, F1=0.528 with 14 DIE correctly identified and 18 still missed. The decision tree captures more information of higher risk, information from attributes like ASCITES, ALBUMIN, and SGOT so it identifies more DIE cases. Because of this the sensitivity increases and a few LIVE cases are mistakenly labelled as DIE which shows the normal precision-recall tradeoff. Therefore these baselines show that accuracy alone is not reliable to check for performance under class imbalance and evaluation for the ensembles will therefore emphasise Recall(die), supported by F1 and PRC Area, with ZeroR and J48 serving as the baseline reference points.

2. Yes, ensemble methods fit this dataset well. The dataset is small with 155 cases and imbalanced with only about 21% DIE. ZeroR had Recall(die)=0.00, J48 with Recall(die)=0.438 but still missed 18 DIE cases. Ensemble methods target this gap. The strongest single baseline is a pruned decision tree which still shows high variance. Bagging helps reduce that variance by training many trees on bootstrap samples and averaging their votes, so predictions become steadier and DIE cases that one tree misses can be caught by others. Random Forest is Bagging but with each split it looks at a random subset of features which lowers correlation between trees and usually improves generalisation on mixed clinical variables and missing values. AdaBoost also helps because it reweights the patients that were misclassified, pushing later trees to focus on the harder DIE cases. The risk with boosting is overfitting if the number of iterations is set too high, so it should be used with moderation. The main problem is class imbalance. Bagging and Random Forest somewhat preserve the original class ratio and hence may still lean toward LIVE, but that does not make ensemble methods unsuitable. It means they should be correctly judged and tuned for the goal which is selecting models by Recall(die) which is supported by F1 or PRC Area and adjusting the probability threshold toward DIE if needed, and if recall remains low, apply class weights or a balanced resample. With these settings, Bagging and Random Forest can be tried first and AdaBoost with proper parameter tuning.

3.

Method	Iterations	Accuracy	Recall (die)	Precision (die)	F1 (die)	PRC Area (die)	TP_die	FN_die
Bagging + J48	5	82.58%	0.438	0.609	0.509	0.603	14	18
Bagging + J48	10	83.87%	0.438	0.667	0.528	0.618	14	18
Bagging + J48	100	83.87%	0.438	0.667	0.528	0.606	14	18
Bagging + J48	200	84.52%	0.438	0.700	0.538	0.617	14	18
Bagging + J48	500	83.23%	0.438	0.636	0.519	0.610	14	18
Bagging + J48	1000	83.23%	0.438	0.636	0.519	0.607	14	18

Bagging was run in Weka (meta classifier) with J48 as the base learner and iterations = 5, 10, 100, 200, 500, 1000 at default settings. Recall(die) stayed at 0.438 in every run (TP=14, FN=18). Bagging mainly reduces variance by averaging many similar trees, with the default probability threshold and a small, imbalanced dataset, the same DIE cases cross the decision boundary each time. The stability improves but the decision boundary does not expand to catch extra DIE cases unless the threshold is adjusted. Because bagging mainly reduces variance but not bias, the averaged tree predictions stay similar with the default 0.5 threshold the decision boundary remains almost unchanged. As false positives on LIVE changed,

Precision(die) changed and F1 and Accuracy shifted accordingly. At 5 iterations, more noisy predictions created 9 false positives with Precision(die)=0.609. Around 200 iterations averaging is strongest, false positives drop to 6 with Precision(die)=0.700, F1=0.538, Accuracy=84.52%. At 500, 1000 averaging adds little new information and false positives increase to 8, so precision and F1 decreased slightly. This is diminishing returns after enough bags. PRC Area(die) is consistent across iterations because bagging smooths predictions without changing class ordering. Bagging is suitable here because it stabilises predictions and improves precision, F1, and accuracy at the same recall. 200 iterations provides the best overall balance and is the recommended setting, as it achieves the highest precision and F1 at the same recall and the highest accuracy.

4.

Method	Iterations	Accuracy	Recall (die)	Precision (die)	F1 (die)	PRC Area (die)	TP_die	FN_die
AdaBoostM1	5	80.65%	0.500	0.533	0.516	0.513	16	16
AdaBoostM1	10	82.58%	0.531	0.586	0.557	0.569	17	15
AdaBoostM1	100	81.29%	0.563	0.545	0.554	0.488	18	14
AdaBoostM1	200	81.29%	0.594	0.543	0.567	0.477	19	13
AdaBoostM1	500	81.94%	0.563	0.563	0.563	0.485	18	14
AdaBoostM1	1000	83.23%	0.563	0.600	0.581	0.482	18	14
RandomForest	5	81.94%	0.313	0.625	0.417	0.603	10	22
RandomForest	10	83.87%	0.438	0.667	0.528	0.616	14	18
RandomForest	100	85.16%	0.469	0.714	0.566	0.651	15	17
RandomForest	200	85.16%	0.438	0.737	0.549	0.641	14	18
RandomForest	500	85.16%	0.469	0.714	0.566	0.652	15	17
RandomForest	1000	85.16%	0.469	0.714	0.566	0.644	15	17

AdaBoostM1 slowly increases Recall(die) as iterations increase peaking at 0.594 at 200 iterations with TP_die=19 and FN_die=13. This is because boosting reweights misclassified cases so later learners focus on the harder DIE cases moving the decision boundary to catch more positives. Precision depends on how many LIVE cases are pulled across that boundary, so F1 rises when recall improves without too many new false positives with the best F1 here at 1000 iterations. RandomForest improves overall separation by averaging many relatively independent models, as iterations increase, Recall(die) rises from 0.313 to 0.469 while precision stays consistent, with the best balance near 100 iterations, highest accuracy 85.16%, F1 0.566, and PRC Area(die) 0.651. Results then reduce showing diminishing returns. Compared to Bagging AdaBoostM1 is the best choice when the priority is maximising DIE recall because it catches more DIE cases as explained before. RandomForest gives the best overall accuracy and stable precision. AdaBoostM1 with 200 iterations is preferred when the priority is avoiding missed DIE cases and RandomForest at 100 iterations is better if a balance of accuracy, precision and recall is preferred.

5.

Method	Composition	Accuracy	Recall	Precision	F1 (die)	PRC Area (die)	TP_die	FN_die
Vote	Diverse: OneR + J48 + NaiveBayes + MLP (ReplaceMissingValues)	86.45%	0.563	0.720	0.632	0.675	18	14
Vote	ZeroR only	79.35%	0.000	0.000*	0.000*	0.196	0	32

The Vote ensemble method with several models like OneR, J48, NaiveBayes, MLP clearly outperforms Vote with just ZeroR across all metrics. ZeroR predicts only LIVE so it achieves 79.35% accuracy from class imbalance and misses every DIE case which is why Recall(die)=0.00, TP=0 and FN=32. The Vote with multiple models averages probabilities from complementary learners like OneR captures simple rules, J48 models interactions, NaiveBayes provides probabilistic smoothing, and MLP adds non-linear patterns so the combined prediction of all of these is better and more robust. This ensemble increases Recall(die) to 0.563 by identifying 18 DIE cases while also keeping false positives moderate which raises Precision(die) to 0.720 and F1 to 0.632 with PRC Area(die)=0.675 which confirms better ranking of DIE compared to LIVE.

Compared with previous ensembles AdaBoostM1 reached the highest recall of 0.594 at 200 iterations because boosting reweights mistakes towards DIE cases which are hard to identify, but its precision and accuracy were lower whereas RandomForest gave strong balance at 100 iterations where Accuracy= 85.16%, F1=0.566, Recall=0.469). Overall Vote with multiple models provides better performance with highest accuracy and F1 with good recall, while AdaBoostM1 at 200 iterations is better when the sole priority is maximising DIE recall. Slight preprocessing was needed here, ReplaceMissingValues was used when adding MLP to Vote method to handle missing data correctly under crossvalidation.

6. In this scenario for this dataset Recall for the DIE class which is also the True Positive Rate is the primary and most meaningful metric. Recall is also called Sensitivity, and it is the standard choice when missing a DIE case matters more than incorrectly classifying a LIVE case as DIE. It directly captures the goal of minimising missed DIE cases by measuring the proportion of actual DIE patients the model correctly identifies. $\text{Recall(die)} = \text{TP_die} / (\text{TP_die} + \text{FN_die})$. This puts classifying DIE first even if it leads to some LIVE patients incorrectly classified as DIE. In contrast, Precision focuses on different errors that are less critical here. A higher Recall means fewer false negatives which is exactly what the task asks to minimise. Accuracy is not suitable here because the data are imbalanced and a model can score high accuracy by predicting LIVE most of the time while failing to detect DIE at all. Metrics like ROC AUC and Accuracy can look strong under imbalance but they would still miss DIE so they are not the primary choice.

7.

Method	Setup	Accuracy	Recall (die)	Precision (die)	F1 (die)	PRC Area (die)	TP_die	FN_die
MultilayerPerceptron	Filtered[ReplaceMissingValues] + MLP	83.87%	0.531	0.630	0.576	0.609	17	15
Bagging + J48	200 iterations	84.52%	0.438	0.700	0.538	0.617	14	18
AdaBoostM1	200 iterations	81.29%	0.594	0.543	0.567	0.477	19	13
RandomForest	100 iterations	85.16%	0.469	0.714	0.566	0.651	15	17
Vote (with multiple models)	OneR + J48 + NaiveBayes + MLP	86.45%	0.563	0.720	0.632	0.675	18	14

A single MultilayerPerceptron was trained with missing values handled inside each CV fold giving an Accuracy of 83.87%, Recall(die) 0.531 and F1 0.576. Compared with ensembles, AdaBoostM1 gives the highest Recall(die) 0.594 because boosting reweights mistakes so later rounds focus on harder DIE cases, which reduces false negatives. RandomForest averages many relatively independent trees, which gives higher precision and accuracy with decent recall, here Accuracy 85.16%, F1 0.566, PRC Area 0.651. The Vote ensemble combined OneR, J48, Naive Bayes and MLP so its averaged probabilities are well calibrated so it achieves the best overall balance with Accuracy 86.45% and F1 0.632 at Recall(die) 0.563. The MLP is between these as it improves on Bagging and RandomForest for recall but below AdaBoost on recall and Vote on F1 and accuracy, which is expected on this small imbalanced dataset where ensembles reduce variance and combine complementary patterns. Here accuracy alone is not reliable under class imbalance. A classifier can score high accuracy while missing DIE cases entirely, as seen with ZeroR earlier. The meaningful primary measure here is Recall because it directly relates to the goal of minimising missed DIE cases with F1(die) and PRC Area(die) as supporting measures to show the precision recall trade off.

8. Yes ensemble methods should be considered effective data mining methods. Ensembles combine multiple models so their independent errors cancel out and their complementary strengths add up which gives lower variance, better generalisation, and more reliable class probabilities on small, noisy, and imbalanced clinical data. Bagging and RandomForest reduce variance by averaging diverse trees whereas AdaBoost reweights mistakes so later rounds focus on cases which are hard to detect. Vote combines different kinds of models, so simple rules, decision trees, probabilistic models, and neural networks each add information that the others might miss. Some limits of ensembles are that they require proper

crossvalidation, enough diversity among base models, and they can show diminishing returns as size grows and they may be less interpretable than a single tree. Despite these limits, when judged by Recall(die) rather than accuracy, ensembles meet the goal more reliably than a single model, so they are recommended.

3.3 Part 3: Data Mining

Introduction

The dataset, IMDB-movie-data.csv contains nearly 5,043 films with attributes including genre, content rating, runtime, year, votes, revenue, critic reviews, and IMDb score. Techniques like classification, clustering, association rule mining, attribute selection visualisation were used to identify what most influences success on IMDb (rating, popularity, engagement) and to find out golden nuggets” from the data.

Question / Hypothesis

The research question was which features most strongly influence success on IMDb (ratings, revenue, audience popularity)? I hypothesised that Action/Drama achieve higher ratings, PG/PG-13 outperform R-rated titles and higher votes and critic reviews correlate with success. To test this, I used supervised models to predict rating class, unsupervised clustering to reveal market segments, Apriori to capture categorical co-occurrences between genre and rating and attribute selection to isolate non-redundant predictors.

Data Preparation and Pre-processing

I removed non-predictive text/ID fields (Rank, Title, Description, Director, Actors) to cut noise. Missing Revenue and Metascore were imputed with ReplaceMissingValues to keep all 5,043 rows. Genre was converted to nominal using *StringToNominal* and commas were stripped from Votes so it's numeric. IMDb score was discretised into RatingClass (Low ≤ 6.0 , Medium 6.1–7.0, High ≥ 7.1) to support classification and Apriori. Numeric features (Revenue, Votes, Runtime, Metascore) were normalised to [0-1] so that distance based methods are not dominated by large scales. Impute to cleaning to discretising then normalising the data prevents error propagation and ensures consistent inputs. Final features (Genre, Year, Runtime, Content Rating, Votes, Revenue, Metascore, RatingClass) balance signal and interpretability.

Data Mining Techniques (methods, parameters, and rationale)

Classification: RandomForest (10-fold CV) was chosen because it handles mixed types and non-linear effects and is robust to noise and it achieved $\text{corr} = 0.881$, $\text{MAE} = 0.0116$, $\text{RMSE} = 0.0267$, showing the numeric rating is predictable from engagement and critic signals. REPTree ($\text{corr} = 0.7199$) provided a fast, interpretable baseline, IB1/1-NN ($\text{corr} = 0.5192$) was a distance-based comparator. Their weaker results justify the ensemble choice.

Clustering: SimpleKMeans ($k=4$, Euclidean, max 500 iters; converged in 7, WC-SSE=12,862.58) was applied to reveal structure in numeric space because it is efficient and produces readable centroids. The clusters mapped to clear segments like a blockbuster group (Action/Adventure, PG-13, highest gross/votes), critically strong dramas (longer duration, top-bin scores), mid-tier comedies (R, moderate engagement), and lower-performing thrillers (R, lowest engagement). This helped learn more about the market segments without labels.

Association: Apriori was used within a FilteredAssociator keeping only categorical fields with $\text{minSupport}=0.80$, $\text{metric}=\text{lift}$ greater then or equal to 0.9 because it gives transparent if-then rules. At this strict support, the strongest findings were broad, stable patterns like $\text{Horror}=f \Rightarrow \text{Mystery}=f$, $\text{conf}=0.92$, $\text{lift}=1.02$) this avoids low-support, incorrect rules keeping the submission defensible, while acknowledging that more specific genre and rating rules would require a lower support threshold.

Attribute Selection: CfsSubsetEval + BestFirst was used (forward, $\text{stale}=5$) to identify a compact, non-redundant predictor set because CFS rewards attributes that add unique signal. Against the popularity target which is `movie_facebook_likes` it selected `num_critic_for_reviews` and `num_voted_users` (subset

merit=0.694), directly supporting the claim that critical visibility and audience engagement are the most influential drivers.

Results and Analysis

RandomForest giving strong correlation shows that ratings are predictable from engagement and critic features the weaker baselines confirm the added value of an ensemble on this mixed dataset. K-Means produced four clear segments which are blockbuster cluster with Action/Adventure, PG-13, highest gross and engagement (votes, reviews), a critically strong drama cluster (higher duration, top-bin IMDb scores), a mid-tier comedy cluster (R, moderate gross, lower engagement) and a lower-performing thriller cluster (R, lowest gross/engagement). Apriori at strict support returned broad co-absences rather than high precision genre and rating rules showing that genre composition is uneven. Attribute selection isolates critic reviews and user votes as the two most informative predictors of popularity.

Conclusions

Overall the analysis shows that IMDb success is driven by audience engagement (votes) and critical visibility (critic reviews, Metascore) but also supported by genre/audience fit (Action/Drama, PG-13). RandomForest worked best because it models non-linear, mixed-type interactions robustly but it is less transparent than a single tree. REPTree added interpretability but simplified complex patterns while IB1 highlighted sensitivity to scaling. K-Means efficiently showed the numeric structure but assumes roughly spherical clusters. Apriori produced if-then patterns which are meaningful but they require careful support tuning. CFS gave a non-redundant predictor set (critics + votes), explaining popularity with minimal attributes. These techniques combined provide solid data driven information about films that earn both public attention and critical coverage, especially in accessible genres, consistently achieve stronger IMDb outcomes.

3.4 Part 4: Summary and Self-reflection

Team Summary:

Working together on this assignment was highly effective. We planned on how to solve each part together and agreed upon the objectives and metrics up front and kept a shared log of runs so every result was reproducible and easy to compare. In Part 1 we aligned on preprocessing choices and consistent crossvalidation settings and in Part 2 we agreed that Recall for the DIE class would be the primary metric for the given dataset and worked on running the ensemble experiments together and in Part 3 we alternated exploration and verification so only findings that were well supported and ground were kept and now in Part 4 we are reviewing what was done during to finish this assignment from the start. Working as a team also let us combine complementary strengths that we shared in Weka, interpreting the model and results and writing the report. When there was disagreement between us upon something we tested alternatives and chose the option that we thought best matched the question and the given goal. This helped us make proper and clear decisions and led to fewer errors and faster iteration. We also knew good communication was crucial so we made sure we always had short checkins and a simple checklist for each run mentioning the classifier, parameters, timestamp and a shared table template for metrics. Cross checking each others work also helped us catch silly errors and helped us make our justifications better. Overall, Working together definitely led to us answering the questions better and more effectively, we finished with a clearer idea of how to state the problem, choose the right models, and support our conclusions with the correct results.

Individual Reflection (s4068533 – Yadav Ranganathan):

Across both assignments my understanding moved from running algorithms to reasoning about them. I can now explain why bagging reduces variance, why boosting increases sensitivity, and many more concepts about data mining. More importantly, I learned to match relevant metrics to the objective of the problem like for high risk positives Recall is the right primary measure which is supported by F1 and PRC Area and to justify threshold choices when recall must be prioritised. I am also a bit more systematic about validation by including preprocessing inside folds, using fixed seeds for fairness and documenting parameters and knowing when to stop when returns diminish. I also feel more confident about deciding if a real problem suits data mining by checking label quality, feature signal, sample size, and whether the evaluation metric reflects the real decision risk. I also kept a run log of models, settings, and seeds, and used the same results table each time, which made my conclusions easy to check and explain. Overall, I now feel ready to tackle new datasets from start to the end and frame the problem clearly, choose metrics that match the risk, select and validate models, and explain the results and trade-offs that are observed.

Individual Reflection (s3859590 – Prathibha Magesh):

I learnt a lot doing this assignment especially turning results into defensible decisions. Early on I compared models mainly by accuracy but now I start from the outcome that matters and pick metrics accordingly. Hands on work with trees, neural networks, and ensembles helped me clearly understand the bias and variance trade-off and how ensemble size hits diminishing returns and why probability calibration matters under imbalance. I am more careful about experiment design, changing one thing at a time, recording it, and verifying it with cross-validation. I also learnt to communicate tradeoffs clearly like raising Recall(die) may increase false positives. Pairing claims with confusion matrix counts and precision recall measures made our golden nuggets credible and useful. These skills strengthen my ability to scope, execute, and explain data mining solutions for real world problems.